Milestone 1 | Interview Questions

Section 1: Software Industry & Career Basics

Question 1: Understanding Software Teams

Scenario: You joined a new company as a Junior Developer. Your manager asks you to fix a bug, but the designer says the button color must match the brand guidelines, and the QA person says the feature isn't working as expected.
Question: Who should you talk to first and why?

Answer: You should first talk to the QA person to understand what exactly is not working. Then, you should fix the bug in your code. After fixing it, coordinate with the designer to ensure the button color matches brand guidelines. This shows you understand that fixing functionality comes before styling, and you know how to work with different team members.

Question 2: Career Path Selection

Scenario: Your friend is confused. He likes designing how websites look, but he also enjoys writing logic for login systems.
Question: Which career path might suit him - Frontend, Backend, or Full Stack? Explain why.

Answer: Full Stack Development would suit him best because:

- Frontend work involves designing how websites look (using HTML, CSS, JavaScript)
- Backend work involves writing logic for systems like login, databases
- A Full Stack Developer does both, which matches his interests in both areas

Question 3: Problem-Solving vs Shortcuts

Scenario: Two freshers join a company. Fresher A spends time understanding why code works. Fresher B just copies code from the internet to complete tasks quickly.
Question: After 6 months, who will be more successful and why?

Answer: Fresher A will be more successful because:

- Understanding WHY code works builds strong fundamentals
- Copying without understanding means you cannot solve new problems
- In interviews and real work, you need problem-solving skills, not just copying ability
- Companies value developers who can think and adapt, not just copy-paste

Question 4: AI in Development

Scenario: Your colleague says, "AI tools like ChatGPT can write code, so developers will lose jobs soon."
Question: Is this statement correct? Explain your view.

Answer: This statement is NOT fully correct because:

- AI is a helping tool, not a replacement for developers
- AI can generate basic code, but cannot understand business requirements
- Developers are still needed to review, test, and fix AI-generated code
- AI cannot make decisions, handle edge cases, or debug complex problems
- Companies need humans to lead projects, make architectural decisions, and ensure quality

Question 5: Skills for Success

Scenario: In an interview, the interviewer asks: "What matters more - knowing 10 programming languages or having strong problem-solving skills?"
Question: What is the best answer and why?

Answer: Strong problem-solving skills matter more because:

- Programming languages are tools; problem-solving is the core skill
- Languages change over time, but logical thinking remains the same
- If you can solve problems well, learning new languages becomes easy
- Companies hire for your ability to think and solve, not just memorize syntax
- A good problem solver can adapt to any technology

Section 2: Programming Fundamentals

Question 6: Why Programming Languages Exist

Scenario: A non-technical person asks you: "Why can't we just talk to computers in English? Why do we need programming languages?"
Question: Explain in simple words.

Answer: Computers only understand 0s and 1s (binary), not English. Humans cannot easily write in 0s and 1s. Programming languages are a middle ground - they are:

- Easy for humans to read and write
- Can be translated to 0s and 1s for computers to understand
- They give precise, step-by-step instructions because computers cannot guess or assume like humans

Question 7: Compiled vs Interpreted Languages

Scenario: Your teammate says, "Java code is compiled, and JavaScript code is interpreted."
Question: What is the difference between compiled and interpreted? Give one advantage of each.

Answer:

Compiled languages (like Java, C++):

- Code is converted to machine code BEFORE running
- Advantage: Runs faster because translation is already done

Interpreted languages (like Python, JavaScript):

- Code is translated line-by-line WHILE running
- Advantage: Easier to test and debug because you can run code immediately without waiting for compilation

Question 8: Understanding Variables

Scenario: You are explaining code to a junior. You have this pseudocode:
age = 25
name = "Rahul"
isStudent = true

Question: Explain what each line does and why we use variables.

Answer:

- `age = 25` stores the number 25 in a container called "age"
- `name = "Rahul"` stores the text "Rahul" in a container called "name"
- `isStudent = true` stores a true/false value in "isStudent"

We use variables because:

- Programs need to store and remember data
- Variables let us use the same data in multiple places
- We can change values when needed (like updating age on a birthday)

Question 9: Loops Concept

Scenario: You need to send a birthday message to 100 students in your database.
Question: Why would you use a loop instead of writing the same code 100 times?

Answer: Using a loop is better because:

- Writing the same code 100 times is time-wasting and error-prone
- A loop lets you repeat the same task automatically
- If you need to change the message, you change it in ONE place, not 100 places
- Code becomes shorter, cleaner, and easier to maintain
- Example: `for each student in database, send birthday message`

Question 10: Functions and Reusability

Scenario: In your project, you calculate tax at 5 different places. You wrote the calculation formula 5 times.
Question: What problem will you face? How can functions help?

Answer: Problems:

- If tax rate changes, you must update 5 different places
- High chance of making mistakes or missing one place
- Code becomes long and messy

Solution - Use a Function:

```
function calculateTax(amount):
    return amount * 0.18
```

Now call this function at all 5 places. If tax changes, update only the function, and all 5 places automatically get the new calculation.

Question 11: Data Types

Scenario: You are building a form where users enter:

- Their name
- Their age
- Whether they agree to terms and conditions

Question: What data type would you use for each field and why?

Answer:

- **Name:** Text/String - because names are letters, not numbers
- **Age:** Number/Integer - because age is a numeric value we might calculate with
- **Agree to T&C:** Boolean (true/false) - because user either agrees or doesn't, only two options

Question 12: Syntax and Keywords

Scenario: A beginner writes this code:

```
function = 10
if = "hello"
```

Question: What is wrong with this code?

Answer: The words `function` and `if` are **keywords** (reserved words) in programming languages. Keywords have special meanings and cannot be used as variable names.

Correct approach:

```
value = 10
message = "hello"
```

Use meaningful names that are NOT keywords.

Section 3: Problem Solving & Algorithms

Question 13: Breaking Down a Problem

Scenario: Your manager asks you to build a feature: "Users should be able to reset their password."
Question: Break this into smaller steps (algorithm).

Answer:

1. User clicks "Forgot Password"
2. System asks for registered email
3. User enters email
4. System checks if email exists in database
5. If email exists, send reset link to email
6. If email does NOT exist, show error message
7. User clicks reset link from email
8. System shows "Enter New Password" form
9. User enters and confirms new password
10. System updates password in database
11. Show success message

Question 14: Input-Output Identification

Scenario: You need to write a program that checks if a student passed or failed. Pass marks = 40.
Question: What are the inputs and outputs?

Answer:

- **Input:** Student's marks (example: 55)
- **Output:** Result - "Pass" or "Fail"

Logic:

```
If marks >= 40:
   Output "Pass"
Else:
   Output "Fail"
```

Question 15: Real-Life Algorithm

Scenario: Write an algorithm for: "Making a cup of tea"

Answer:

Step 1: Boil water in a kettle
Step 2: Take a cup
Step 3: Add tea leaves/tea bag to cup
Step 4: Add sugar (optional, based on preference)

Step 5: Pour boiled water into cup
Step 6: Wait for 2 minutes
Step 7: Add milk (optional)
Step 8: Stir well
Step 9: Tea is ready
This shows you understand: sequence, optional steps, and waiting/timing.

Question 16: Pseudocode for Discount

Scenario: A shop gives 10% discount if purchase is above ₹500.
Question: Write pseudocode to calculate final price.

Answer:

```
Input: originalPrice
If originalPrice > 500:
   discount = originalPrice * 0.10
   finalPrice = originalPrice - discount
Else:
   finalPrice = originalPrice
Output: finalPrice
```

Question 17: Identifying Edge Cases

Scenario: You wrote code to divide two numbers entered by the user.
Question: What edge case should you handle? Write pseudocode.

Answer: Edge case: User might enter 0 as the second number, which causes division by zero error.
Pseudocode:

```
Input: num1, num2
If num2 == 0:
   Output "Cannot divide by zero"
Else:
   result = num1 / num2
   Output result
```

Question 18: SDLC Understanding

Scenario: Your company is building a food delivery app.
Question: Map the SDLC phases to this project with examples.

Answer:

1. **Planning:** Decide to build a food delivery app, identify target users
2. **Design:** Design how app will look, plan database for restaurants, orders, users

3. **Development:** Developers write code for frontend (app screens) and backend (order processing)
4. **Testing:** QA team tests if users can place orders, payments work, etc.
5. **Deployment:** App is released on Google Play Store / App Store
6. **Maintenance:** Fix bugs reported by users, add new features like "track delivery"

Question 19: Agile vs Waterfall (Awareness)

Scenario: In an interview, you're asked: "Have you heard of Agile?"
Question: What would you say?

Answer: "Yes, Agile is a way of working where teams build software in small parts (called sprints), test quickly, and improve based on feedback. It's different from Waterfall, where you complete each phase fully before moving to the next. Agile is popular because it allows faster delivery and adapting to changes."

Question 20: Algorithm for Login

Scenario: Write an algorithm for a login system.

Answer:
Step 1: User enters email and password
Step 2: System checks if email exists in database
Step 3: If email does NOT exist:
    Show "User not found"
    Stop
Step 4: If email exists:
    Fetch stored password for that email
Step 5: Compare entered password with stored password
Step 6: If passwords match:
    Login successful, redirect to dashboard
Step 7: If passwords do NOT match:
    Show "Incorrect password"

Section 4: Internet, Browsers & Full Stack

Question 21: Client-Server Model

Scenario: You open Amazon.com on your browser.
Question: Explain what happens using the client-server model.

Answer:
1. **Client (your browser)** sends a request to Amazon's server asking for the homepage
2. **Server** receives the request, prepares the homepage HTML/CSS/JS
3. **Server** sends the response back to your browser

4. **Browser** receives the response and displays the Amazon homepage on your screen

Client = your browser (makes requests)
Server = Amazon's computer (sends responses)

Question 22: DNS Explanation

Scenario: Your friend asks, "What is DNS?"
Question: Explain DNS in simple language with an example.

Answer: DNS is like a phonebook for the internet.

- When you type `www.google.com`, your computer doesn't understand this name
- DNS converts `www.google.com` into an IP address like `142.250.183.46`
- Your browser then uses this IP address to connect to Google's server

Without DNS, you would have to remember and type IP addresses (numbers) instead of easy names like google.com.

Question 23: HTTP vs HTTPS

Scenario: You're building a login page for your website.
Question: Should you use HTTP or HTTPS? Why?

Answer: You must use HTTPS because:

- HTTPS encrypts data between browser and server
- Login pages send sensitive information like passwords
- HTTP sends data in plain text, which hackers can read
- HTTPS ensures data is secure and cannot be stolen easily
- Browsers show a warning if you use HTTP for login pages

Rule: Always use HTTPS for pages with passwords, payments, personal data.

Question 24: What is Frontend?

Scenario: In an interview, you're asked: "What does a frontend developer do?"
Question: Give a clear answer with examples.

Answer: A frontend developer builds what users see and interact with on a website or app.
Responsibilities:

- Create page layouts using HTML
- Style pages (colors, fonts, spacing) using CSS
- Add interactions (button clicks, form validations) using JavaScript
- Make sure the website works on mobile and desktop
- Ensure good user experience

Example: The login form you see, buttons you click, animations - all are frontend work.

Question 25: What is Backend?

Scenario: A non-technical person asks: "What happens in the backend?"
Question: Explain with a simple example.

Answer: Backend is the hidden part of a website that users don't see. It handles:

- **Data storage:** Saving user information in a database
- **Logic:** Checking if password is correct during login
- **Security:** Making sure only authorized users can access data
- **APIs:** Sending data to the frontend

Example: When you log in to Instagram:

- Frontend shows the login form
- Backend checks if your username and password are correct in the database
- If correct, backend sends your profile data to frontend to display

Question 26: Full Stack Developer Role

Scenario: Your company is hiring a Full Stack Developer.
Question: What skills should this person have?

Answer: A Full Stack Developer should know:
Frontend skills:

- HTML, CSS, JavaScript
- Frameworks like React

Backend skills:

- Server-side language like Node.js, Python, Java
- Database knowledge (SQL, MongoDB)
- API development

Other skills:

- Understanding of how frontend and backend connect
- Problem-solving and debugging
- Version control (Git)

They should be able to build a complete feature from user interface to database.

Question 27: HTML, CSS, JavaScript Roles

Scenario: You're teaching a beginner about web pages.
Question: Explain the role of HTML, CSS, and JavaScript with a real-life analogy.

Answer: Think of a house:

- **HTML** = Structure (walls, rooms, doors) - It defines WHAT is on the page (headings, buttons, images)
- **CSS** = Decoration (paint, furniture, colors) - It defines HOW things look (colors, sizes, spacing)

- **JavaScript =** Functionality (electricity, plumbing) - It defines what happens when you INTERACT (button clicks, form submissions)

Example:

- HTML creates a button
- CSS makes it blue and round
- JavaScript makes it do something when clicked

Question 28: Cloud Computing Benefits

Scenario: A startup is deciding whether to buy their own servers or use cloud services like AWS.
Question: What would you recommend and why?

Answer: Recommend using cloud (AWS/Azure/Google Cloud) because:
Advantages:

- **Cost-effective:** Pay only for what you use, no upfront server cost
- **Scalable:** If users increase, easily add more resources
- **Accessible:** Access from anywhere with internet
- **Maintenance-free:** Cloud provider handles hardware, updates, security

For a startup:

- Limited budget, so cloud is cheaper
- User base might grow quickly, cloud can scale
- Team can focus on building product, not managing servers

Question 29: IaaS vs PaaS vs SaaS

Scenario: In an interview, you're asked to explain cloud service types.
Question: Explain IaaS, PaaS, and SaaS with examples.

Answer:
IaaS (Infrastructure as a Service):

- You rent basic computing resources (servers, storage)
- You install and manage everything else
- Example: AWS EC2 - you get a virtual machine, you install your own software

PaaS (Platform as a Service):

- You get a ready platform to build and deploy apps
- No need to manage servers or infrastructure
- Example: Heroku - you just upload your code, platform handles rest

SaaS (Software as a Service):

- You use ready-made software over the internet
- No installation, no development needed
- Example: Gmail, Netflix - just use the software

Question 30: Browser Rendering

Scenario: You write HTML, CSS, and JavaScript code in VS Code and open the file in a browser.
Question: What does the browser do to display your webpage?

Answer:
Step 1: Browser reads your HTML file
Step 2: It builds the DOM (Document Object Model) - a structure of your page
Step 3: Browser reads CSS and applies styling to the DOM
Step 4: Browser executes JavaScript to add interactions
Step 5: Browser displays the final styled and interactive page on your screen
If you change code and refresh, browser repeats these steps with the new code.

Question 31: Request-Response Flow

Scenario: You click "Sign Up" button on a website.
Question: Describe the complete request-response flow.

Answer:

1. **Browser (client)** sends HTTP/HTTPS request to server with form data (name, email, password)
2. **Request travels** through the internet to the server's IP address (found using DNS)
3. **Server receives** the request
4. **Server checks** if email already exists in database
5. If email is new:
    - Server saves user data in database
    - Server sends response: "Sign up successful"
6. If email exists:
    - Server sends response: "Email already registered"
7. **Browser receives** response
8. **Browser displays** success or error message to user

Question 32: Why Use VS Code?

Scenario: A beginner asks, "Can't I just write code in Notepad?"
Question: Explain why VS Code (or IDEs) are better than Notepad.

Answer:
Yes, you CAN write code in Notepad, but VS Code is much better because:

- **Syntax Highlighting:** Code is colored, making it easier to read
- **Auto-completion:** Suggests code as you type, saving time
- **Error Detection:** Shows errors before you run the code
- **Extensions:** Add tools for formatting, debugging, Git
- **Integrated Terminal:** Run code without leaving the editor

- **File Management:** Work with multiple files and folders easily

Notepad is plain text - no help, no colors, no suggestions.

Question 33: IP Address Purpose

Scenario: Your friend says, "Why do we need IP addresses if we have domain names like google.com?"
Question: Explain why IP addresses are necessary.

Answer:

- Computers and servers communicate using **numbers (IP addresses)**, not names
- Domain names like `google.com` are for humans - easy to remember
- DNS translates domain names to IP addresses behind the scenes
- **Without IP addresses,** computers wouldn't know WHERE to send data
- **Without domain names,** users would have to remember numbers like `142.250.183.46` instead of `google.com`

Both are needed: Domain for humans, IP for computers.

Question 34: Frontend-Backend Communication

Scenario: You're building a "Submit Feedback" feature.
Question: Explain how frontend and backend work together.

Answer:
Frontend (what user sees):

- Shows a form with name, email, feedback text boxes
- User fills the form and clicks "Submit"
- Frontend sends this data to backend using an HTTP request

Backend (hidden logic):

- Receives the data
- Validates: checks if email format is correct, fields are not empty
- Saves feedback in the database
- Sends response back: "Feedback submitted successfully"

Frontend receives response:

- Shows success message to user
- Clears the form

Connection: Frontend and backend communicate through APIs (requests and responses).

Question 35: DOM Understanding

Scenario: An interviewer asks, "What is DOM in simple terms?"
Question: Give a beginner-friendly explanation.

Answer:

DOM (Document Object Model) is a representation of your HTML page that JavaScript can work with.

Think of it as a tree:

- HTML page is broken into elements (headings, buttons, divs)
- Each element is a "node" in the tree
- JavaScript can access these nodes and change them

Example:

<h1 id="title">Welcome</h1>

JavaScript can use DOM to:

- Find this heading: `document.getElementById("title")`
- Change text: `title.innerHTML = "Hello"`
- Change color: `title.style.color = "blue"`

Without DOM, JavaScript cannot interact with HTML elements.

Bonus Scenario-Based Questions

Question 36: Debugging Mindset

Scenario: Your code worked yesterday, but today it's showing an error. You didn't change anything.

Question: What steps would you take to debug?

Answer:

1. **Stay calm** - panicking doesn't help
2. **Read the error message** - it often tells you what's wrong
3. **Check recent changes** - did you update any library, package, or dependency?
4. **Reproduce the error** - can you make it happen again?
5. **Google the error** - someone else likely faced this
6. **Check browser console** - for frontend errors
7. **Use console.log()** - print values to see where code breaks
8. **Ask a colleague** - fresh eyes can spot issues
9. **Check internet/server** - maybe external service is down

Never assume "nothing changed" - environment, dependencies, or services may have changed.

Question 37: Code Review Culture

Scenario: Your senior reviews your code and gives feedback. You feel the code works fine.

Question: How should you respond?

Answer:

Correct approach:

- Thank them for the review
- Ask questions to understand their suggestions

- Consider their experience - they might see issues you missed
- Make changes if feedback improves code quality, security, or readability
- If you disagree, respectfully explain your reasoning and discuss

Wrong approach:

- Getting defensive
- Ignoring feedback because "code works"
- Arguing without understanding

Remember: Code reviews help you learn and improve. Working code ≠ good code. There's always room for better practices, performance, security.

Question 38: Time Estimation

Scenario: Your manager asks, "How long will it take to build a login page?"
Question: How should you answer?

Answer:
Don't say: "2 hours" (too specific without thinking)
Better approach:

1. **Clarify requirements:**
    - Just frontend or backend too?
    - Email login or social login (Google, Facebook)?
    - Forgot password feature needed?
    - Testing included?
2. **Break into tasks:**
    - Frontend form: 2 hours
    - Backend API: 3 hours
    - Database setup: 1 hour
    - Testing: 2 hours
    - Total: 8 hours
3. **Add buffer:** Unexpected issues happen, so add 20-30% extra time
4. **Communicate:** "Based on these features, it will take approximately 1.5 days, but I'll update you if I face blockers."

Shows: You think before committing, ask questions, and plan properly.

Question 39: Learning New Technology

Scenario: Your company is moving from JavaScript to TypeScript. You don't know TypeScript.
Question: What approach would you take?

Answer:

1. **Don't panic** - learning is part of a developer's life
2. **Understand WHY** - ask why company is switching (helps motivation)
3. **Find official resources** - TypeScript docs, tutorials
4. **Start small** - learn basics first (types, interfaces)
5. **Practice** - convert small JavaScript code to TypeScript

6. **Ask team** - learn from colleagues already using it
7. **Build something** - small project to apply learning
8. **Stay consistent** - 1 hour daily is better than 10 hours once

Mindset: Technologies change, but your ability to LEARN is the real skill.

Question 40: Production Bug Priority

Scenario: Users report that the "Add to Cart" button is not working on your e-commerce website. At the same time, your manager asks you to add a new feature.
Question: What should you prioritize and why?

Answer:
Priority: Fix the "Add to Cart" bug FIRST
Reasons:

- This is a **production bug** - affecting real users right now
- Users cannot buy products = company loses money
- New features can wait, but bugs directly impact business
- User trust and experience are at risk

Approach:

1. Inform manager about the critical bug
2. Fix the bug immediately
3. Test thoroughly
4. Deploy the fix
5. Then resume work on the new feature

Shows: You understand business impact, prioritization, and urgency handling.